

UERS
PROGRAMAÇÃO ORIENTADA A OBJETOS

FILIPPE TORMES MENGUE
GABRIEL GARCIA VIEIRA

**DOCUMENTAÇÃO TÉCNICA DO SISTEMA DE
CONTROLE DE ESTOQUE**

Projeto em C++ com API HTTP, PostgreSQL e Drogon

Trabalho apresentado à disciplina de Programação Orientada a Objetos, como parte da avaliação do projeto prático desenvolvido em C++.

Professor: André Borin Soares

Porto Alegre
2026

Sumário

1	Introdução	3
2	Tecnologias Utilizadas	3
3	Objetivo do Sistema	3
4	Organização Geral dos Arquivos	3
5	Arquitetura do Projeto	4
6	Fluxo de Inicialização	5
7	Configuração da Aplicação	5
7.1	Variáveis de Ambiente	5
7.2	Leitura do arquivo .env	6
8	Modelo de Banco de Dados	6
8.1	Tabela estoque_items	6
8.2	Tabela estoque_inventory_movements	7
8.3	Índices	7
9	Rotas da API	7
9.1	GET /health	8
9.2	GET /api/movements	8
9.3	GET /api/items	9
9.4	GET /api/items/{code}	9
9.5	POST /api/items	10
9.6	PATCH /api/items/{code}	10
9.7	DELETE /api/items/{code}	11
9.8	POST /api/items/{code}/inbound	11
9.9	POST /api/items/{code}/outbound	11
10	Modelos de Dados do Código	11
11	Camada HTTP: InventoryController	12
11.1	Leitura e validação de JSON	12
11.2	Conversão para JSON	12

11.3 Tratamento centralizado de erros	12
12 Camada de Serviço: InventoryService	13
12.1 ensureSchemaSync	13
12.2 listMovements	13
12.3 listItems	13
12.4 getItemByCode	13
12.5 createItem	13
12.6 updateItem	14
12.7 registerInbound	14
12.8 registerOutbound	14
12.9 deleteItem	14
12.10loadItemDetailsByCode	14
13 Tipos de Item e Conceitos de Orientação a Objetos	14
14 Validações de Negócio	15
15 Tratamento de Erros	15
16 Controle de Concorrência	16
17 CORS	16
18 Build com CMake	16
19 Execução	17
20 Docker	17
21 Exemplos de Requisições	17
21.1 Criar item	17
21.2 Listar itens	18
21.3 Pesquisar item	18
21.4 Registrar entrada	18
21.5 Registrar saída	18
22 Resumo das Principais Decisões Técnicas	18
23 Conclusão	19

1 Introdução

O projeto desenvolvido consiste em uma API para controle de estoque, implementada em C++ com o framework Drogon. A ideia principal foi construir um backend capaz de cadastrar itens, consultar dados do estoque, atualizar informações e registrar entradas e saídas, mantendo também um histórico das movimentações realizadas.

Durante o desenvolvimento, o código foi separado em partes com responsabilidades diferentes. O controller ficou responsável pelas rotas e pelas respostas HTTP, enquanto a classe de serviço ficou com as regras de negócio e com as consultas ao banco de dados. O PostgreSQL foi utilizado para guardar os itens e as movimentações, e o próprio sistema executa o script de criação das tabelas quando é iniciado.

2 Tecnologias Utilizadas

- **C++20**: linguagem principal do projeto.
- **Drogon**: framework web usado para criar a API HTTP e trabalhar com corrotinas.
- **PostgreSQL**: banco de dados relacional utilizado para persistir os itens e movimentações.
- **CMake**: ferramenta de configuração e build do projeto.
- **Docker**: opção de empacotamento para execução em ambiente Linux ou serviços de deploy.
- **JSON**: formato utilizado no corpo das requisições e respostas da API.

3 Objetivo do Sistema

Na prática, o sistema foi pensado para resolver as operações principais de um controle de estoque simples. Com a API, é possível:

- cadastrar itens de estoque;
- listar todos os itens cadastrados;
- pesquisar itens por código ou nome;
- consultar os detalhes de um item pelo código;
- atualizar informações cadastrais do item;
- registrar entrada de estoque;
- registrar saída de estoque;
- impedir saída maior que a quantidade disponível;
- excluir itens;
- manter histórico das movimentações.

4 Organização Geral dos Arquivos

Os arquivos principais do projeto foram divididos por responsabilidade:

- **src/main.cpp**: ponto de entrada da aplicação. Carrega as configurações, conecta ao banco, executa o schema inicial, configura CORS e inicia o servidor Drogon.

- `AppConfig.hpp` e `AppConfig.cpp`: cuidam da configuração da aplicação. Nesses arquivos ficam a leitura do `.env`, das variáveis de ambiente e a montagem da string de conexão com o PostgreSQL.
- `InventoryController.hpp` e `InventoryController.cpp`: definem e implementam as rotas HTTP. Também fazem a leitura do JSON recebido, montam as respostas e tratam erros vindos da camada de serviço.
- `InventoryService.hpp` e `InventoryService.cpp`: concentram as regras de negócio do estoque, como cadastro, atualização, entrada, saída, exclusão, busca e validação de saldo.
- `InventoryModels.hpp`: reúne as estruturas usadas para transportar dados entre as camadas do projeto.
- `ApiError.hpp`: define uma exceção própria para retornar erros da API com status HTTP específico.
- `ItemTypeProfile.hpp` e `ItemTypeProfile.cpp`: implementam a parte de tipos de item, usando interface, herança e polimorfismo para diferenciar produto e material.
- `sql/schema.sql`: contém a criação das tabelas, restrições e índices usados pelo PostgreSQL.
- `CMakeLists.txt`: configura a compilação do executável `inventory_api`.
- `Dockerfile`: permite compilar e executar a aplicação em container.
- `docs/backend-class-diagram.mmd`: guarda o diagrama de classes do backend em formato Mermaid.

5 Arquitetura do Projeto

O projeto segue uma separação simples em camadas:

- **Camada HTTP**: representada por `InventoryController`, recebe as requisições e devolve respostas JSON.
- **Camada de serviço**: representada por `InventoryService`, concentra a lógica de negócio.
- **Camada de dados**: formada pelo PostgreSQL e pelos comandos SQL executados pelo serviço.
- **Camada de configuração**: representada por `AppConfig`, responsável por carregar parâmetros de execução.

Essa divisão facilita manutenção, pois a regra de negócio não fica misturada diretamente com as rotas HTTP. Por exemplo, o controller sabe como receber uma requisição, mas quem decide se uma saída de estoque é permitida é o serviço.

6 Fluxo de Inicialização

O arquivo `src/main.cpp` contém a função `main`, que inicia todo o sistema. O fluxo principal é:

1. carregar configurações por meio de `AppConfig::loadFromEnvironment()`;
2. aplicar configurações de SSL do PostgreSQL, quando informadas;
3. calcular a quantidade de threads;
4. imprimir um resumo da inicialização no terminal;
5. criar um cliente temporário de banco para inicialização;
6. executar o schema SQL por meio de `InventoryService::ensureSchemaSync()`;
7. registrar o listener HTTP no host e porta configurados;
8. registrar a política global de CORS;
9. criar o cliente PostgreSQL principal usado pelo Drogon;
10. definir número de threads e nível de log;
11. iniciar o loop de eventos do Drogon com `drogon::app().run()`.

Esse processo faz com que a API já prepare o banco antes de começar a atender requisições.

7 Configuração da Aplicação

A configuração é representada pelas estruturas `AppConfig`, `DatabaseConfig` e `CorsConfig`. O sistema tenta ler primeiro variáveis de ambiente reais e, caso elas não existam, procura os mesmos nomes dentro do arquivo `.env` localizado na raiz do projeto.

7.1 Variáveis de Ambiente

Variável	Valor padrão	Descrição
APP_HOST	0.0.0.0	Endereço em que a API ficará escutando.
APP_PORT	2020	Porta padrão da aplicação.
PORT	vazio	Alternativa usada por plataformas de deploy. Tem prioridade sobre APP_PORT.
APP_THREADS	0	Quantidade de threads. Quando é zero, usa a quantidade disponível do processador.
POSTGRES_HOST	127.0.0.1	Host do banco PostgreSQL.
POSTGRES_PORT	5432	Porta do PostgreSQL.
POSTGRES_DB	inventory_api	Nome do banco.
POSTGRES_USER	postgres	Usuário do banco.
POSTGRES_PASSWORD	vazio	Senha do banco.
POSTGRES_CONNECTIONS	4	Tamanho do pool de conexões.

POSTGRES_CLIENT_NAME	default	Nome do cliente de banco registrado no Drogon.
POSTGRES_FAST_CLIENT	false	Define se o cliente Drogon será do tipo fast client.
POSTGRES_CHARSET	UTF8	Codificação usada na conexão.
POSTGRES_TIMEOUT	-1.0	Timeout do cliente de banco.
POSTGRES_AUTO_BATCH	false	Configuração de auto batch do Drogon.
POSTGRES_SSLMODE	vazio	Modo SSL do PostgreSQL, quando necessário.
CORS_ALLOWED_ORIGIN	*	Origem permitida para requisições CORS.
CORS_ALLOWED_METHODS	GET, POST, PATCH, DELETE, OPTIONS	Métodos HTTP permitidos.
CORS_ALLOWED_HEADERS	Content-Type, Authorization	Cabeçalhos permitidos.
CORS_MAX_AGE	86400	Tempo de cache da resposta de preflight.

7.2 Leitura do arquivo .env

O arquivo `src/api/AppConfig.cpp` possui uma função que lê o `.env` linha por linha. Linhas vazias e comentários são ignorados. Para cada linha no formato `CHAVE=VALOR`, o código remove espaços desnecessários e também remove aspas simples ou duplas ao redor do valor, caso existam.

Isso permite configurar o projeto localmente sem gravar senhas diretamente no código fonte.

8 Modelo de Banco de Dados

O banco possui duas tabelas principais: `estoque_items` e `estoque_inventory_movements`.

8.1 Tabela `estoque_items`

Essa tabela armazena o cadastro atual dos itens de estoque.

Campo	Tipo	Descrição
id	bigserial	Chave primária gerada automaticamente.
code	varchar(50)	Código único do item.
item_type	varchar(20)	Tipo do item. Aceita apenas <code>product</code> ou <code>material</code> .
name	varchar(255)	Nome do item.

description	text	Descrição do item.
information_link	text	Link de referência ou informação complementar.
quantity	integer	Quantidade atual em estoque. Não pode ser negativa.
unit_price	double precision	Valor unitário. Não pode ser negativo.
location	varchar(255)	Localização física do item no estoque.
created_at	timestamptz	Data de criação do registro.
updated_at	timestamptz	Data da última atualização.

8.2 Tabela estoque_inventory_movements

Essa tabela guarda o histórico de entradas e saídas de estoque.

Campo	Tipo	Descrição
id	bigserial	Chave primária da movimentação.
item_id	bigint	Referência ao item movimentado. Possui <code>on delete cascade</code> .
operation	varchar(20)	Tipo da operação. Aceita <code>inbound</code> ou <code>outbound</code> .
quantity	integer	Quantidade movimentada. Deve ser maior que zero.
note	text	Observação da movimentação.
created_at	timestamptz	Data de criação da movimentação.

8.3 Índices

O schema cria dois índices:

- `idx_estoque_items_name_lower`: ajuda em buscas pelo nome do item em letras minúsculas.
- `idx_estoque_inventory_movements_item_id`: melhora consultas das movimentações por item.

9 Rotas da API

As rotas são declaradas em `include/http/InventoryController.hpp` usando as macros do Drogon. A implementação fica em `src/http/InventoryController.cpp`.

Método	Rota	Descrição
GET	<code>/health</code>	Verifica se a API está ativa.
GET	<code>/api/movements</code>	Lista todo o histórico de movimentações de estoque.

GET	/api/items	Lista todos os itens cadastrados. Pode receber o filtro search .
GET	/api/items/{code}	Consulta os detalhes de um item pelo código.
POST	/api/items	Cria um novo item de estoque.
PATCH	/api/items/{code}	Atualiza parcialmente os dados cadastrais de um item.
DELETE	/api/items/{code}	Remove um item pelo código.
POST	/api/items/{code}/inbound	Registra entrada de estoque para um item.
POST	/api/items/{code}/outbound	Registra saída de estoque para um item.

9.1 GET /health

Retorna uma resposta simples indicando que o processo da API está funcionando.

Resposta:

```
{
  "status": "ok",
  "service": "inventory_api"
}
```

9.2 GET /api/movements

Lista o histórico completo de movimentações. A consulta faz join entre a tabela de movimentações e a tabela de itens, retornando também código, tipo e nome do item.

Resposta:

```
{
  "movements": [
    {
      "id": 1,
      "item_id": 1,
      "item_code": "D001",
      "item_type": "product",
      "item_name": "Wireless Mouse",
      "operation": "inbound",
      "quantity": 10,
      "note": "Initial registration.",
      "created_at": "2026-07-02T10:00:00Z"
    }
  ],
  "total": 1
}
```

9.3 GET /api/items

Lista os itens cadastrados. Sem filtro, retorna todos os itens ordenados por nome e código. Com o parâmetro `search`, filtra por código ou nome usando `ilike`.

Exemplo:

```
GET /api/items?search=mouse
```

Resposta:

```
{
  "items": [
    {
      "id": 1,
      "code": "D001",
      "item_type": "product",
      "name": "Wireless Mouse",
      "quantity": 10,
      "unit_price": 89.9,
      "total_value": 899.0,
      "location": "Shelf A1",
      "updated_at": "2026-07-02T10:00:00Z"
    }
  ],
  "total": 1
}
```

9.4 GET /api/items/{code}

Busca um item pelo código e retorna também o histórico de movimentações desse item. Se o código não existir, retorna erro 404.

Resposta:

```
{
  "item": {
    "id": 1,
    "code": "D001",
    "item_type": "product",
    "name": "Wireless Mouse",
    "description": "Mouse for office use.",
    "information_link": "https://example.com/mouse",
    "quantity": 10,
    "unit_price": 89.9,
    "total_value": 899.0,
    "location": "Shelf A1",
    "created_at": "2026-07-02T10:00:00Z",
    "updated_at": "2026-07-02T10:00:00Z",
    "movements": []
  }
}
```

9.5 POST /api/items

Cria um item novo. O código deve ser único e o tipo deve ser `product` ou `material`. Se a quantidade inicial for maior que zero, o sistema registra automaticamente uma movimentação de entrada com a observação `Initial registration`.

Corpo da requisição:

```
{
  "code": "D001",
  "item_type": "product",
  "name": "Wireless Mouse",
  "description": "Mouse for office use.",
  "information_link": "https://example.com/mouse",
  "quantity": 10,
  "unit_price": 89.9,
  "location": "Shelf A1"
}
```

Resposta de sucesso: código HTTP 201.

```
{
  "message": "Item created successfully.",
  "item": {
    "id": 1,
    "code": "D001",
    "item_type": "product",
    "name": "Wireless Mouse",
    "quantity": 10
  }
}
```

9.6 PATCH /api/items/{code}

Atualiza parcialmente os dados do item. Pelo menos um campo precisa ser enviado. A quantidade não é alterada nessa rota, pois entradas e saídas de estoque possuem rotas próprias.

Campos aceitos:

- `item_type`
- `name`
- `description`
- `information_link`
- `unit_price`
- `location`

Exemplo:

```
{
  "description": "Ergonomic wireless mouse.",
}
```

```
"unit_price": 99.5,  
"location": "Shelf B2"  
}
```

9.7 DELETE /api/items/{code}

Remove o item informado pelo código. Como a tabela de movimentações usa `on delete cascade`, o histórico relacionado ao item também é removido automaticamente pelo banco.

Resposta:

```
{  
  "message": "Item deleted successfully."  
}
```

9.8 POST /api/items/{code}/inbound

Registra uma entrada de estoque. A quantidade deve ser maior ou igual a 1, e a observação é obrigatória. A operação ocorre dentro de transação.

Corpo da requisição:

```
{  
  "quantity": 5,  
  "note": "Weekly replenishment."  
}
```

O serviço atualiza a quantidade do item e grava uma linha na tabela de movimentações com `operation = inbound`.

9.9 POST /api/items/{code}/outbound

Registra uma saída de estoque. Antes de baixar a quantidade, o sistema verifica a quantidade atual. Se a saída for maior do que o saldo disponível, a API retorna erro 409.

Corpo da requisição:

```
{  
  "quantity": 3,  
  "note": "Separated for customer order."  
}
```

O serviço atualiza a quantidade do item e grava uma linha na tabela de movimentações com `operation = outbound`.

10 Modelos de Dados do Código

Os modelos ficam em `include/api/InventoryModels.hpp`. Eles funcionam como DTOs, ou seja, estruturas simples para transportar dados entre camadas.

Modelo	Uso
InventoryMovementDto	Representa uma movimentação de um item específico no detalhe do item.
InventoryMovementHistoryDto	Representa uma movimentação na listagem geral, incluindo dados do item.
InventoryItemSummaryDto	Representa o item de forma resumida na listagem.
InventoryItemDetailsDto	Representa o item completo, incluindo descrição, link, datas e movimentações.
CreateItemDto	Dados necessários para criar um item.
UpdateItemDto	Dados opcionais para atualizar parcialmente um item.
StockChangeRequestDto	Dados usados nas operações de entrada e saída de estoque.

11 Camada HTTP: InventoryController

O controller é a camada que conversa diretamente com o cliente HTTP. Ele tem quatro responsabilidades principais:

- declarar quais métodos atendem cada rota;
- ler e validar o corpo JSON das requisições;
- chamar os métodos da camada de serviço;
- converter o resultado para JSON e definir o status HTTP correto.

11.1 Leitura e validação de JSON

O controller possui funções auxiliares para ler campos obrigatórios e opcionais:

- `requiredStringField`: exige que o campo exista e seja string.
- `optionalStringField`: aceita campo ausente, mas se existir deve ser string.
- `requiredIntField`: exige número inteiro.
- `requiredDoubleField`: exige número numérico.
- `optionalDoubleField`: aceita campo ausente, mas se existir deve ser numérico.
- `requestJsonBody`: verifica se o corpo enviado é um JSON válido.

Quando uma validação falha, é lançada uma `ApiError` com status 400.

11.2 Conversão para JSON

O controller também possui funções `toJson` para transformar cada DTO em resposta JSON. Isso deixa a regra de negócio livre de detalhes de apresentação HTTP.

11.3 Tratamento centralizado de erros

A função `template` `executeWithErrorHandling` envolve a execução das rotas. Ela captura:

- `ApiError`: retorna o status definido pela própria exceção;
- `DrogonDbException`: registra o erro no log e retorna erro 500;
- `std::exception`: retorna erro interno 500.

Esse ponto é importante porque evita repetir `try/catch` completo em todas as rotas.

12 Camada de Serviço: InventoryService

A classe `InventoryService` concentra as regras de negócio e o acesso ao banco. Ela recebe um `DbClientPtr` no construtor, permitindo executar consultas pelo cliente PostgreSQL configurado no Drogon.

12.1 `ensureSchemaSync`

Lê o arquivo `sql/schema.sql`, separa os comandos SQL e executa cada um deles. O código possui uma separação própria de comandos para evitar quebrar SQL incorretamente quando houver ponto e vírgula dentro de strings.

12.2 `listMovements`

Executa uma consulta com `inner join` entre itens e movimentações. O resultado é ordenado do mais recente para o mais antigo pelo campo `m.id desc`.

12.3 `listItems`

Lista os itens cadastrados. Quando não existe filtro, busca todos. Quando existe `search`, monta um filtro com `%texto%` e usa `ilike` em `code` e `name`. Também calcula `total_value` diretamente no SQL por meio de `quantity * unit_price`.

12.4 `getItemByCode`

Valida se o código não está vazio e chama `loadItemDetailsByCode`. Se o item não for encontrado, a função interna lança erro 404.

12.5 `createItem`

Cria um item novo dentro de uma transação. O fluxo é:

1. validar campos obrigatórios;
2. normalizar o tipo do item;
3. verificar se já existe item com o mesmo código;
4. inserir o registro na tabela `estoque_items`;

5. se a quantidade inicial for maior que zero, inserir movimentação inicial;
6. retornar o item completo.

Se ocorrer erro de chave duplicada, o serviço retorna erro 409.

12.6 `updateItem`

Atualiza os campos cadastrais do item. A função não permite chamada vazia: pelo menos um campo precisa ser enviado. O serviço carrega o item atual, combina os dados atuais com o patch recebido, valida o resultado e executa o `update`.

12.7 `registerInbound`

Registra entrada de estoque. O serviço abre uma transação, bloqueia o item com `for update`, soma a quantidade no saldo atual e registra a movimentação.

O bloqueio `for update` evita inconsistência caso duas operações tentem alterar o mesmo item ao mesmo tempo.

12.8 `registerOutbound`

Registra saída de estoque. O fluxo é parecido com a entrada, mas antes de atualizar o saldo o serviço verifica se existe quantidade suficiente.

Se a quantidade solicitada for maior que o estoque atual, o retorno é erro 409 com a mensagem informando que não há quantidade suficiente.

12.9 `deleteItem`

Remove o item pelo código. Se nenhuma linha for afetada, retorna erro 404. As movimentações relacionadas são removidas pelo próprio PostgreSQL por causa da restrição `on delete cascade`.

12.10 `loadItemDetailsByCode`

É uma função interna usada por várias operações. Ela busca os dados completos do item e depois consulta suas movimentações. As datas são convertidas para o formato UTC ISO, como `YYYY-MM-DDTHH:MM:SSZ`.

13 Tipos de Item e Conceitos de Orientação a Objetos

O projeto possui uma pequena hierarquia de classes para representar tipos de item. A classe `ItemTypeProfile` é abstrata e define dois métodos virtuais puros:

- `storageValue`: valor salvo no banco;

- `displayLabel`: rótulo de exibição.

As classes `ProductItemTypeProfile` e `MaterialItemTypeProfile` herdam de `ItemTypeProfile`. A função `makeItemTypeProfile` recebe o texto informado, normaliza para minúsculo e retorna a implementação correta usando `std::unique_ptr<ItemTypeProfile>`.

Isso demonstra:

- **Interface:** `ItemTypeProfile` define um contrato.
- **Herança:** `ProductItemTypeProfile` e `MaterialItemTypeProfile` reaproveitam o contrato.
- **Polimorfismo:** o código trabalha com ponteiro para a interface, mas executa o método da classe real.

14 Validações de Negócio

As principais validações estão em `InventoryService.cpp`:

Validação	Regra
Texto obrigatório	Código, nome, descrição, link, localização e observação não podem ficar vazios.
Tipo do item	Só aceita <code>product</code> ou <code>material</code> .
Preço unitário	Não pode ser negativo.
Quantidade no cadastro	Pode ser zero, mas não pode ser negativa.
Quantidade em movimentação	Deve ser maior ou igual a 1.
Código duplicado	Retorna erro 409 se o código já existir.
Saída de estoque	Não permite retirar mais itens do que a quantidade disponível.
Atualização vazia	Não permite PATCH sem nenhum campo.
Item inexistente	Retorna erro 404.

15 Tratamento de Erros

O sistema usa a classe `ApiError`, que herda de `std::runtime_error` e guarda um código HTTP. Assim, a camada de serviço consegue lançar erros de negócio sem depender diretamente da montagem de resposta HTTP.

Status	Situação
400	JSON inválido, campo ausente, tipo incorreto, quantidade inválida, preço negativo ou tipo de item inválido.
404	Item não encontrado.
409	Código duplicado ou tentativa de saída maior que o estoque disponível.
500	Erro de banco de dados ou erro interno inesperado.

O formato padrão de erro é:

```
{
  "error": "Mensagem explicando o problema."
}
```

16 Controle de Concorrência

As operações de entrada e saída de estoque usam transações e bloqueio de linha. Nas duas rotas, o serviço executa uma consulta com `for update` antes de alterar o saldo.

Esse detalhe é importante porque evita que duas requisições simultâneas leiam a mesma quantidade antiga e atualizem o estoque de forma incorreta. Na saída, o bloqueio também garante que a validação de saldo seja feita sobre uma quantidade consistente.

17 CORS

O arquivo `main.cpp` registra duas configurações relacionadas a CORS:

- uma regra de `PreRoutingAdvice` para responder requisições `OPTIONS` com status 204;
- uma regra de `PreSendingAdvice` para adicionar os cabeçalhos CORS em todas as respostas.

Por padrão, a origem permitida é `*`. Caso seja necessário restringir para um frontend específico, basta configurar `CORS_ALLOWED_ORIGIN`.

18 Build com CMake

O arquivo `CMakeLists.txt` define o projeto `InventoryApi`, exige `C++20`, procura o pacote `Drogon` e cria o executável `inventory_api`.

```
cmake -S . -B build-api -G "MinGW Makefiles" \
  -DCMAKE_BUILD_TYPE=Release

cmake --build build-api -j 8
```

Em compiladores GNU ou Clang, o projeto ativa as opções `-Wall`, `-Wextra` e `-pedantic`. No Windows, o CMake também copia DLLs necessárias para a pasta do executável.

19 Execução

Depois de compilado, o executável pode ser iniciado diretamente. A aplicação tenta carregar o arquivo `.env` da raiz do projeto.

```
.\build-api\inventory_api.exe
```

Ao iniciar, o sistema mostra no terminal um resumo com URL, endereço de bind, quantidade de threads, dados do banco, cliente PostgreSQL, SSL, charset e configuração de CORS.

20 Docker

O Dockerfile usa dois estágios:

- **builder**: instala ferramentas e dependências de compilação, copia o código e compila o projeto.
- **runtime**: instala apenas o necessário para executar, copia o binário e o diretório sql.

A imagem final executa com um usuário próprio chamado `appuser`, o que é melhor do que rodar a aplicação como `root`. Também existe um `HEALTHCHECK` que chama a rota `/health`.

```
docker build -t inventory-api .

docker run --name inventory-api \
  --env-file .env \
  -p 2020:2020 \
  inventory-api
```

21 Exemplos de Requisições

21.1 Criar item

```
curl -X POST http://localhost:2020/api/items \
  -H "Content-Type: application/json" \
  -d '{
    "code": "D001",
    "item_type": "product",
    "name": "Wireless Mouse",
    "description": "Mouse for office use.",
    "information_link": "https://example.com/mouse",
    "quantity": 10,
    "unit_price": 89.9,
    "location": "Shelf A1"
  }'
```

21.2 Listar itens

```
curl http://localhost:2020/api/items
```

21.3 Pesquisar item

```
curl "http://localhost:2020/api/items?search=mouse"
```

21.4 Registrar entrada

```
curl -X POST http://localhost:2020/api/items/D001/inbound \
-H "Content-Type: application/json" \
-d '{
  "quantity": 5,
  "note": "Reposicao semanal."
}'
```

21.5 Registrar saída

```
curl -X POST http://localhost:2020/api/items/D001/outbound \
-H "Content-Type: application/json" \
-d '{
  "quantity": 3,
  "note": "Separado para pedido."
}'
```

22 Resumo das Principais Decisões Técnicas

- A API foi feita em C++20 com Drogon para aproveitar corrotinas e rotas HTTP de forma direta.
- A regra de negócio foi concentrada no `InventoryService`, deixando o controller mais limpo.
- O banco PostgreSQL foi usado por ser relacional e adequado para controlar integridade dos dados.
- O histórico de movimentações fica separado do cadastro do item, preservando rastreabilidade.
- As alterações de estoque usam transações para evitar inconsistência.
- As saídas de estoque validam saldo antes de atualizar a quantidade.
- O schema é aplicado automaticamente na inicialização, simplificando a preparação do ambiente.
- A configuração por `.env` evita dados fixos no código.
- O Dockerfile permite executar o mesmo projeto em ambiente Linux de forma mais padronizada.

23 Conclusão

Com o projeto, foi possível aplicar conceitos de Programação Orientada a Objetos em uma aplicação com uso real de API, banco de dados e separação de responsabilidades. A implementação permite cadastrar, consultar, atualizar, remover e movimentar itens, mantendo o histórico de entradas e saídas para cada produto ou material.

A organização em controller, serviço, modelos, configuração e banco de dados deixou o código mais fácil de entender e de modificar. Para uma próxima etapa, faria sentido adicionar autenticação, paginação nas listagens, testes automatizados, controle de usuários e relatórios de valor total por categoria.